

Laboratorio 6 Parte 1

En este laboratorio, estaremos repasando los conceptos de Generative Adversarial Networks. En la segunda parte nos acercaremos a esta arquitectura a través de buscar generar numeros que parecieran ser generados a mano. Esta vez ya no usaremos versiones deprecadas de la librería de PyTorch, por ende, creen un nuevo virtual env con las librerías más recientes que puedan por favor.

Al igual que en laboratorios anteriores, para este laboratorio estaremos usando una herramienta para Jupyter Notebooks que facilitará la calificación, no solo asegurando que ustedes tengan una nota pronto sino también mostrandoles su nota final al terminar el laboratorio.

De nuevo me discupo si algo no sale bien, seguiremos mejorando conforme vayamos iterando. Siempre pido su comprensión y colaboración si algo no funciona como debería.

Al igual que en el laboratorio pasado, estaremos usando la librería de Dr John Williamson et al de la University of Glasgow, además de ciertas piezas de código de Dr Bjorn Jensen de su curso de Introduction to Data Science and System de la University of Glasgow para la visualización de sus calificaciones.

NOTA: Ahora tambien hay una tercera dependencia que se necesita instalar. Ver la celda de abajo por favor

```
In [1]: # Una vez instalada la librería por favor, recuerden volverla a comentar.  
#!pip install -U --force-reinstall --no-cache https://github.com/johnhw/jhwutils/zip  
#!pip install scikit-image  
#!pip install -U --force-reinstall --no-cache https://github.com/AlbertS789/lautils/
```

```
In [1]: !pip install -U --force-reinstall --no-cache https://github.com/johnhw/jhwutils/zip  
!pip install scikit-image  
!pip install -U --force-reinstall --no-cache https://github.com/AlbertS789/lautils/
```

```

Collecting https://github.com/johnhw/jhwutils/zipball/master
  Downloading https://github.com/johnhw/jhwutils/zipball/master
    - 119.1 kB 10.3 MB/s 0:00:00
  Preparing metadata (setup.py) ... done
Building wheels for collected packages: jhwutils
  Building wheel for jhwutils (setup.py) ... done
  Created wheel for jhwutils: filename=jhwutils-1.3-py3-none-any.whl size=41854 sha2
56=65bbf3bcc69a549478fc90971688c0af02e866d5ab0e73f6fbae128f2cbb0b43
  Stored in directory: /tmp/pip-ephem-wheel-cache-x5tgvs5/wheels/ea/a5/c1/a5a791fc4
679b6b51da9afbde7aaf0a44719030ae5715746d3
Successfully built jhwutils
Installing collected packages: jhwutils
Successfully installed jhwutils-1.3
Requirement already satisfied: scikit-image in /usr/local/lib/python3.12/dist-packag
es (0.25.2)
Requirement already satisfied: numpy>=1.24 in /usr/local/lib/python3.12/dist-package
s (from scikit-image) (2.0.2)
Requirement already satisfied: scipy>=1.11.4 in /usr/local/lib/python3.12/dist-packa
ges (from scikit-image) (1.16.1)
Requirement already satisfied: networkx>=3.0 in /usr/local/lib/python3.12/dist-packa
ges (from scikit-image) (3.5)
Requirement already satisfied: pillow>=10.1 in /usr/local/lib/python3.12/dist-packag
es (from scikit-image) (11.3.0)
Requirement already satisfied: imageio!=2.35.0,>=2.33 in /usr/local/lib/python3.12/d
ist-packages (from scikit-image) (2.37.0)
Requirement already satisfied: tifffile>=2022.8.12 in /usr/local/lib/python3.12/dist
-packages (from scikit-image) (2025.6.11)
Requirement already satisfied: packaging>=21 in /usr/local/lib/python3.12/dist-packa
ges (from scikit-image) (25.0)
Requirement already satisfied: lazy-loader>=0.4 in /usr/local/lib/python3.12/dist-pa
ckages (from scikit-image) (0.4)
Collecting https://github.com/AlbertS789/lautils/zipball/master
  Downloading https://github.com/AlbertS789/lautils/zipball/master
    - 4.2 kB ? 0:00:00
  Preparing metadata (setup.py) ... done
Building wheels for collected packages: lautils
  Building wheel for lautils (setup.py) ... done
  Created wheel for lautils: filename=lautils-1.0-py3-none-any.whl size=2826 sha256=
62b34de8d1f96e20e929ba52a9f6824e40a91b5ad37f02dd4239b96d2573e397
  Stored in directory: /tmp/pip-ephem-wheel-cache-h7ctjjna/wheels/bf/c9/fa/c5e1b05da
8bca40206e1a779b037d0fd93eea00e7fcfa51b6e
Successfully built lautils
Installing collected packages: lautils
Successfully installed lautils-1.0

```

```

In [2]: import numpy as np
import copy
import matplotlib.pyplot as plt
import scipy
from PIL import Image
import os
from collections import defaultdict

#from IPython import display
#from base64 import b64decode

```

```
# Other imports
from unittest.mock import patch
from uuid import getnode as get_mac

from jhwutils.checkarr import array_hash, check_hash, check_scalar, check_string, a
import jhwutils.image_audio as ia
import jhwutils.tick as tick
from lautils.gradeutils import new_representation, hex_to_float, compare_numbers, c

###
tick.reset_marks()

%matplotlib inline
```

In [3]: *# Celda escondida para utilidades necesarias, por favor NO edite esta celda*

Información del estudiante en dos variables

- carne_1 : un string con su carne (e.g. "12281"), debe ser de al menos 5 caracteres.
- firma_mecanografiada_1: un string con su nombre (e.g. "Albero Suriano") que se usará para la declaracion que este trabajo es propio (es decir, no hay plagio)
- carne_2 : un string con su carne (e.g. "12281"), debe ser de al menos 5 caracteres.
- firma_mecanografiada_2: un string con su nombre (e.g. "Albero Suriano") que se usará para la declaracion que este trabajo es propio (es decir, no hay plagio)

```
In [4]: carne_1 = "22075"
        firma_mecanografiada_1 = "Diego Duarte"
        carne_2 = "22155"
        firma_mecanografiada_2 = "Joaquin Campos"
```

```
In [5]: # Deberia poder ver dos checkmarks verdes [0 marks], que indican que su información

        with tick.marks(0):
            assert(len(carne_1)>=5 and len(carne_2)>=5)

        with tick.marks(0):
            assert(len(firma_mecanografiada_1)>0 and len(firma_mecanografiada_2)>0)
```

✓ [0 marks]

✓ [0 marks]

Introducción

Créditos: Esta parte de este laboratorio está tomado y basado en uno de los blogs de Renato Candido, así como las imagenes presentadas en este laboratorio a menos que se indique lo contrario.

Las redes generativas adversarias también pueden generar muestras de alta dimensionalidad, como imágenes. En este ejemplo, se va a utilizar una GAN para generar imágenes de dígitos escritos a mano. Para ello, se entrenarán los modelos utilizando el conjunto de datos MNIST de dígitos escritos a mano, que está incluido en el paquete torchvision.

Dado que este ejemplo utiliza imágenes en el conjunto de datos de entrenamiento, los modelos necesitan ser más complejos, con un mayor número de parámetros. Esto hace que el proceso de entrenamiento sea más lento, llevando alrededor de dos minutos por época (aproximadamente) al ejecutarse en la CPU. Se necesitarán alrededor de cincuenta épocas para obtener un resultado relevante, por lo que el tiempo total de entrenamiento al usar una CPU es de alrededor de cien minutos.

Para reducir el tiempo de entrenamiento, se puede utilizar una GPU si está disponible. Sin embargo, será necesario mover manualmente tensores y modelos a la GPU para usarlos en el proceso de entrenamiento.

Se puede asegurar que el código se ejecutará en cualquier configuración creando un objeto de dispositivo que apunte a la CPU o, si está disponible, a la GPU. Más adelante, se utilizará este dispositivo para definir dónde deben crearse los tensores y los modelos, utilizando la GPU si está disponible.

```
In [6]: import torch
        from torch import nn

        import math
        import matplotlib.pyplot as plt
        import torchvision
        import torchvision.transforms as transforms

        import random
        import numpy as np
```

```
In [7]: seed_ = 111

        def seed_all(seed_):
            random.seed(seed_)
            np.random.seed(seed_)
            torch.manual_seed(seed_)
            torch.cuda.manual_seed(seed_)
            torch.backends.cudnn.deterministic = True

        seed_all(seed_)
```

```
In [8]: device = ""
        if torch.cuda.is_available():
            device = torch.device("cuda")
        else:
            device = torch.device("cpu")
        print(device)
```

cuda

Preparando la Data

El conjunto de datos MNIST consta de imágenes en escala de grises de 28×28 píxeles de dígitos escritos a mano del 0 al 9. Para usarlos con PyTorch, será necesario realizar algunas conversiones. Para ello, se define transform, una función que se utilizará al cargar los datos:

La función tiene dos partes:

- `transforms.ToTensor()` convierte los datos en un tensor de PyTorch.
- `transforms.Normalize()` convierte el rango de los coeficientes del tensor.

Los coeficientes originales proporcionados por `transforms.ToTensor()` varían de 0 a 1, y dado que los fondos de las imágenes son negros, la mayoría de los coeficientes son iguales a 0 cuando se representan utilizando este rango.

`transforms.Normalize()` cambia el rango de los coeficientes a -1 a 1 restando 0.5 de los coeficientes originales y dividiendo el resultado por 0.5. Con esta transformación, el número de elementos iguales a 0 en las muestras de entrada se reduce drásticamente, lo que ayuda en el entrenamiento de los modelos.

Los argumentos de `transforms.Normalize()` son dos tuplas, $(M_1, \dots, M_n)$ y $(S_1, \dots, S_n)$, donde n representa el número de canales de las imágenes. Las imágenes en escala de grises como las del conjunto de datos MNIST tienen solo un canal, por lo que las tuplas tienen solo un valor. Luego, para cada canal i de la imagen, `transforms.Normalize()` resta M_i de los coeficientes y divide el resultado por S_i .

Luego se pueden cargar los datos de entrenamiento utilizando `torchvision.datasets.MNIST` y realizar las conversiones utilizando transform

El argumento `download=True` garantiza que la primera vez que se ejecute el código, el conjunto de datos MNIST se descargará y almacenará en el directorio actual, como se indica en el argumento `root`.

Después que se ha creado `train_set`, se puede crear el cargador de datos como se hizo antes en la parte 1.

Cabe decir que se puede utilizar Matplotlib para trazar algunas muestras de los datos de entrenamiento. Para mejorar la visualización, se puede usar `cmap=gray_r` para invertir el mapa de colores y representar los dígitos en negro sobre un fondo blanco:

Como se puede ver más adelante, hay dígitos con diferentes estilos de escritura. A medida que la GAN aprende la distribución de los datos, también generará dígitos con diferentes estilos de escritura.

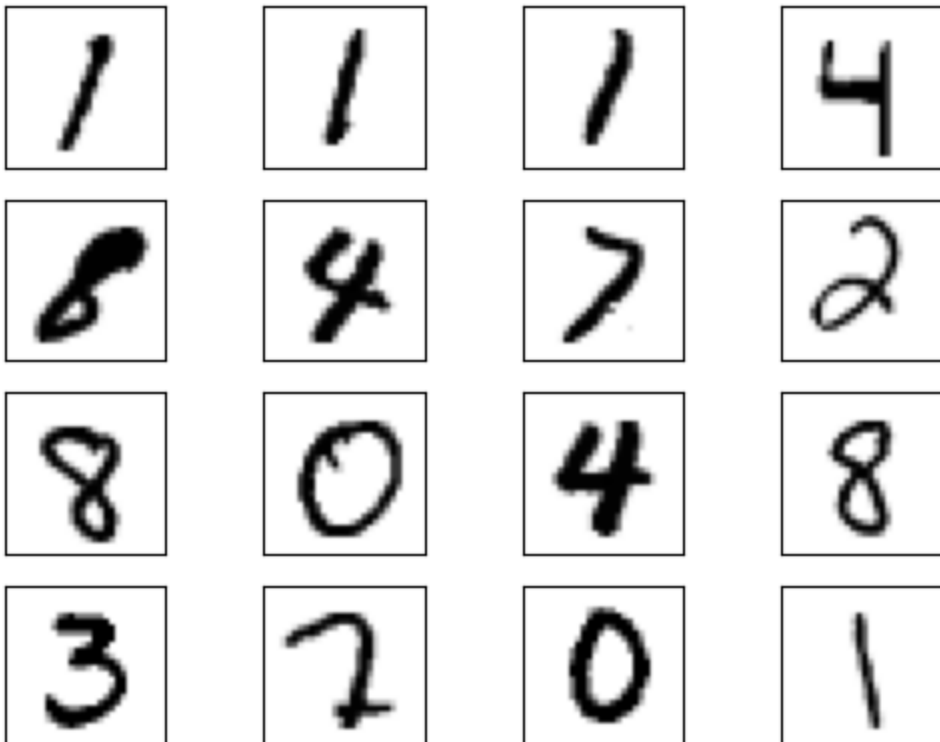
```
In [9]: transform = transforms.Compose(
        [transforms.ToTensor(), transforms.Normalize((0.5,), (0.5,))]
    )
```

```
In [10]: train_set = torchvision.datasets.MNIST(
        root=".", train=True, download=True, transform=transform
    )
```

```
100%|██████████| 9.91M/9.91M [00:02<00:00, 4.56MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 134kB/s]
100%|██████████| 1.65M/1.65M [00:01<00:00, 1.09MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 12.3MB/s]
```

```
In [11]: batch_size = 32
        train_loader = torch.utils.data.DataLoader(
            train_set, batch_size=batch_size, shuffle=True
        )
```

```
In [12]: real_samples, mnist_labels = next(iter(train_loader))
        for i in range(16):
            ax = plt.subplot(4, 4, i + 1)
            plt.imshow(real_samples[i].reshape(28, 28), cmap="gray_r")
            plt.xticks([])
            plt.yticks([])
```



Implementando el Discriminador y el Generador

En este caso, el discriminador es una red neuronal MLP (multi-layer perceptron) que recibe una imagen de 28×28 píxeles y proporciona la probabilidad de que la imagen pertenezca a los datos reales de entrenamiento.

Para introducir los coeficientes de la imagen en la red neuronal MLP, se vectorizan para que la red neuronal reciba vectores con 784 coeficientes.

La vectorización ocurre cuando se ejecuta `.forward()`, ya que la llamada a `x.view()` convierte la forma del tensor de entrada. En este caso, la forma original de la entrada "x" es $32 \times 1 \times 28 \times 28$, donde 32 es el tamaño del batch que se ha configurado. Después de la conversión, la forma de "x" se convierte en 32×784 , con cada línea representando los coeficientes de una imagen del conjunto de entrenamiento.

Para ejecutar el modelo de discriminador usando la GPU, hay que instanciarlo y enviarlo a la GPU con `.to()`. Para usar una GPU cuando haya una disponible, se puede enviar el modelo al objeto de dispositivo creado anteriormente.

Dado que el generador va a generar datos más complejos, es necesario aumentar las dimensiones de la entrada desde el espacio latente. En este caso, el generador va a recibir una entrada de 100 dimensiones y proporcionará una salida con 784 coeficientes, que se organizarán en un tensor de 28×28 que representa una imagen.

Luego, se utiliza la función tangente hiperbólica `Tanh()` como activación de la capa de salida, ya que los coeficientes de salida deben estar en el intervalo de -1 a 1 (por la normalización que se hizo anteriormente). Después, se instancia el generador y se envía a device para usar la GPU si está disponible.

```
In [13]: import torch
import torch.nn as nn

class Discriminator(nn.Module):
    def __init__(self):
        super().__init__()
        self.model = nn.Sequential(
            nn.Linear(784, 1024),
            nn.ReLU(),
            nn.Dropout(0.3),

            nn.Linear(1024, 512),
            nn.ReLU(),
            nn.Dropout(0.3),

            nn.Linear(512, 256),
            nn.ReLU(),
            nn.Dropout(0.3),

            nn.Linear(256, 1),
            nn.Sigmoid()
        )
```

```
def forward(self, x):
    x = x.view(x.size(0), 784)
    output = self.model(x)
    return output
```

```
In [14]: import torch
import torch.nn as nn

class Generator(nn.Module):
    def __init__(self):
        super().__init__()
        self.model = nn.Sequential(
            nn.Linear(100, 256),
            nn.ReLU(),

            nn.Linear(256, 512),
            nn.ReLU(),

            nn.Linear(512, 1024),
            nn.ReLU(),

            nn.Linear(1024, 784),
            nn.Tanh()
        )

    def forward(self, x):
        output = self.model(x)
        output = output.view(x.size(0), 1, 28, 28)
        return output
```

Entrenando los Modelos

Para entrenar los modelos, es necesario definir los parámetros de entrenamiento y los optimizadores como se hizo en la parte anterior.

Para obtener un mejor resultado, se disminuye la tasa de aprendizaje de la primera parte. También se establece el número de épocas en 10 para reducir el tiempo de entrenamiento.

El ciclo de entrenamiento es muy similar al que se usó en la parte previa. Note como se envían los datos de entrenamiento a device para usar la GPU si está disponible

Algunos de los tensores no necesitan ser enviados explícitamente a la GPU con device. Este es el caso de generated_samples, que ya se envió a una GPU disponible, ya que latent_space_samples y generator se enviaron a la GPU previamente.

Dado que esta parte presenta modelos más complejos, el entrenamiento puede llevar un poco más de tiempo. Después de que termine, se pueden verificar los resultados generando algunas muestras de dígitos escritos a mano.


```
In [16]: list_images = []

path_imgs = "./generated_images/"
os.makedirs(path_imgs, exist_ok=True)

seed_all(seed_)

discriminator = Discriminator().to(device=device)
generator = Generator().to(device=device)

lr = 0.0001
num_epochs = 50
loss_function = nn.BCELoss()

optimizer_discriminator = torch.optim.Adam(discriminator.parameters(), lr=lr)
optimizer_generator = torch.optim.Adam(generator.parameters(), lr=lr)

for epoch in range(num_epochs):
    for n, (real_samples, mnist_labels) in enumerate(train_loader):
        # Data for training the discriminator
        real_samples = real_samples.to(device=device)
        real_samples_labels = torch.ones((batch_size, 1)).to(
            device=device
        )
        latent_space_samples = torch.randn((batch_size, 100)).to(
            device=device
        )
        generated_samples = generator(latent_space_samples)
        generated_samples_labels = torch.zeros((batch_size, 1)).to(
            device=device
        )
        all_samples = torch.cat((real_samples, generated_samples))
        all_samples_labels = torch.cat(
            (real_samples_labels, generated_samples_labels)
        )

        # Training the discriminator
        discriminator.zero_grad()
        output_discriminator = discriminator(all_samples)
        loss_discriminator = loss_function(
            output_discriminator, all_samples_labels
        )
        # Aprox dos lineas
        loss_discriminator.backward()
        optimizer_discriminator.step()

        # Data for training the generator
        latent_space_samples = torch.randn((batch_size, 100)).to(
            device=device
        )

        # Training the generator
        generator.zero_grad()
        generated_samples = generator(latent_space_samples)
        output_discriminator_generated = discriminator(generated_samples)
```

```
loss_generator = loss_function(
    output_discriminator_generated, real_samples_labels
)

# Aprox dos lineas para
loss_generator.backward()
optimizer_generator.step()

# Guardamos las imagenes
if epoch % 2 == 0 and n == batch_size - 1:
    generated_samples_detached = generated_samples.cpu().detach()
    for i in range(16):
        ax = plt.subplot(4, 4, i + 1)
        plt.imshow(generated_samples_detached[i].reshape(28, 28), cmap="gray")
        plt.xticks([])
        plt.yticks([])
        plt.title("Epoch "+str(epoch))
    name = path_imgs + "epoch_mnist"+str(epoch)+".jpg"
    plt.savefig(name, format="jpg")
    plt.close()
    list_images.append(name)

# Show Loss
if n == batch_size - 1:
    print(f"Epoch: {epoch} Loss D.: {loss_discriminator}")
    print(f"Epoch: {epoch} Loss G.: {loss_generator}")
```

Epoch: 0 Loss D.: 0.5532951354980469
Epoch: 0 Loss G.: 0.5225616097450256
Epoch: 1 Loss D.: 0.05981234461069107
Epoch: 1 Loss G.: 4.056108474731445
Epoch: 2 Loss D.: 0.07015205174684525
Epoch: 2 Loss G.: 4.422539234161377
Epoch: 3 Loss D.: 0.05965477228164673
Epoch: 3 Loss G.: 8.288107872009277
Epoch: 4 Loss D.: 0.08544750511646271
Epoch: 4 Loss G.: 5.350735187530518
Epoch: 5 Loss D.: 0.07586962729692459
Epoch: 5 Loss G.: 5.374814510345459
Epoch: 6 Loss D.: 0.04852158576250076
Epoch: 6 Loss G.: 2.476529836654663
Epoch: 7 Loss D.: 0.15045909583568573
Epoch: 7 Loss G.: 2.9979143142700195
Epoch: 8 Loss D.: 0.21941521763801575
Epoch: 8 Loss G.: 2.8730859756469727
Epoch: 9 Loss D.: 0.3107296824455261
Epoch: 9 Loss G.: 2.653507709503174
Epoch: 10 Loss D.: 0.4489874243736267
Epoch: 10 Loss G.: 2.2734580039978027
Epoch: 11 Loss D.: 0.2838760018348694
Epoch: 11 Loss G.: 1.846960425376892
Epoch: 12 Loss D.: 0.2540264129638672
Epoch: 12 Loss G.: 2.021592855453491
Epoch: 13 Loss D.: 0.32511305809020996
Epoch: 13 Loss G.: 1.615715742111206
Epoch: 14 Loss D.: 0.4370611310005188
Epoch: 14 Loss G.: 1.4698915481567383
Epoch: 15 Loss D.: 0.4863816499710083
Epoch: 15 Loss G.: 1.3863078355789185
Epoch: 16 Loss D.: 0.41642385721206665
Epoch: 16 Loss G.: 1.2684848308563232
Epoch: 17 Loss D.: 0.40164047479629517
Epoch: 17 Loss G.: 1.281224250793457
Epoch: 18 Loss D.: 0.5096817016601562
Epoch: 18 Loss G.: 1.4334889650344849
Epoch: 19 Loss D.: 0.4131533205509186
Epoch: 19 Loss G.: 1.3610641956329346
Epoch: 20 Loss D.: 0.4794394373893738
Epoch: 20 Loss G.: 1.190194010734558
Epoch: 21 Loss D.: 0.46331849694252014
Epoch: 21 Loss G.: 1.24485445022583
Epoch: 22 Loss D.: 0.4968758225440979
Epoch: 22 Loss G.: 1.1306641101837158
Epoch: 23 Loss D.: 0.5240381360054016
Epoch: 23 Loss G.: 1.3262566328048706
Epoch: 24 Loss D.: 0.6323860287666321
Epoch: 24 Loss G.: 1.0265908241271973
Epoch: 25 Loss D.: 0.5008234977722168
Epoch: 25 Loss G.: 1.0386364459991455
Epoch: 26 Loss D.: 0.4950511157512665
Epoch: 26 Loss G.: 1.2103666067123413
Epoch: 27 Loss D.: 0.5543316006660461
Epoch: 27 Loss G.: 1.1748204231262207

```

Epoch: 28 Loss D.: 0.6287636756896973
Epoch: 28 Loss G.: 0.9960126876831055
Epoch: 29 Loss D.: 0.5333749651908875
Epoch: 29 Loss G.: 1.1109070777893066
Epoch: 30 Loss D.: 0.6001014113426208
Epoch: 30 Loss G.: 1.1485204696655273
Epoch: 31 Loss D.: 0.4577075242996216
Epoch: 31 Loss G.: 1.370239496231079
Epoch: 32 Loss D.: 0.6456784009933472
Epoch: 32 Loss G.: 1.0985748767852783
Epoch: 33 Loss D.: 0.5394789576530457
Epoch: 33 Loss G.: 1.1711459159851074
Epoch: 34 Loss D.: 0.5155236124992371
Epoch: 34 Loss G.: 0.9859165549278259
Epoch: 35 Loss D.: 0.5961123704910278
Epoch: 35 Loss G.: 0.979249119758606
Epoch: 36 Loss D.: 0.5221576690673828
Epoch: 36 Loss G.: 1.1883610486984253
Epoch: 37 Loss D.: 0.6153226494789124
Epoch: 37 Loss G.: 1.2094523906707764
Epoch: 38 Loss D.: 0.48726168274879456
Epoch: 38 Loss G.: 1.1325619220733643
Epoch: 39 Loss D.: 0.516264021396637
Epoch: 39 Loss G.: 0.9681270122528076
Epoch: 40 Loss D.: 0.7419130802154541
Epoch: 40 Loss G.: 0.8866484761238098
Epoch: 41 Loss D.: 0.5622146129608154
Epoch: 41 Loss G.: 1.0338575839996338
Epoch: 42 Loss D.: 0.7196556329727173
Epoch: 42 Loss G.: 1.0549006462097168
Epoch: 43 Loss D.: 0.48883870244026184
Epoch: 43 Loss G.: 1.0464410781860352
Epoch: 44 Loss D.: 0.5313940048217773
Epoch: 44 Loss G.: 0.9016954898834229
Epoch: 45 Loss D.: 0.56558758020401
Epoch: 45 Loss G.: 0.9990841746330261
Epoch: 46 Loss D.: 0.6617120504379272
Epoch: 46 Loss G.: 1.0283892154693604
Epoch: 47 Loss D.: 0.658639669418335
Epoch: 47 Loss G.: 0.9287712574005127
Epoch: 48 Loss D.: 0.6109130382537842
Epoch: 48 Loss G.: 1.0675793886184692
Epoch: 49 Loss D.: 0.5165094137191772
Epoch: 49 Loss G.: 1.1390595436096191

```

```

In [17]: with tick.marks(35):
          assert compare_numbers(new_representation(loss_discriminator), "3c3d", '0x1.333

          with tick.marks(35):
             assert compare_numbers(new_representation(loss_generator), "3c3d", '0x1.8000000

```

```

/usr/local/lib/python3.12/dist-packages/lautils/gradeutils.py:5: UserWarning: Converting a tensor with requires_grad=True to a scalar may lead to unexpected behavior. Consider using tensor.detach() first. (Triggered internally at /pytorch/torch/csrc/autograd/generated/python_variable_methods.cpp:835.)
  num = float(num)

```

✓ [35 marks]

✓ [35 marks]

Validación del Resultado

Para generar dígitos escritos a mano, es necesario tomar algunas muestras aleatorias del espacio latente y alimentarlas al generador.

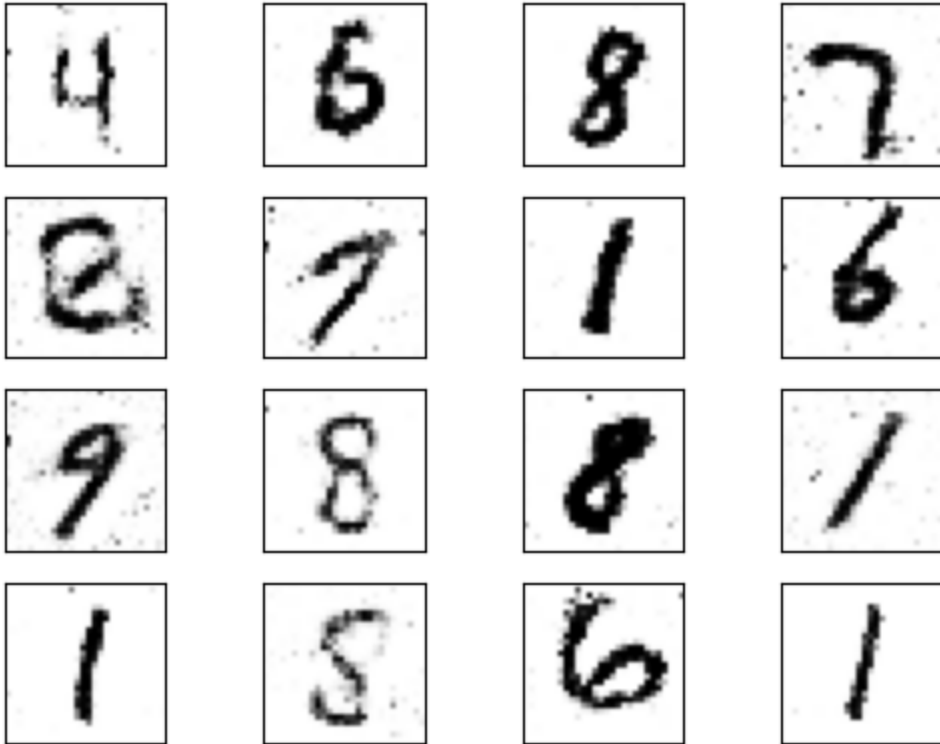
Para trazar `generated_samples`, es necesario mover los datos de vuelta a la CPU en caso de que estén en la GPU. Para ello, simplemente se puede llamar a `.cpu()`. Como se hizo anteriormente, también es necesario llamar a `.detach()` antes de usar Matplotlib para trazar los datos.

La salida debería ser dígitos que se asemejen a los datos de entrenamiento. Después de cincuenta épocas de entrenamiento, hay varios dígitos generados que se asemejan a los reales. Se pueden mejorar los resultados considerando más épocas de entrenamiento. Al igual que en la parte anterior, al utilizar un tensor de muestras de espacio latente fijo y alimentarlo al generador al final de cada época durante el proceso de entrenamiento, se puede visualizar la evolución del entrenamiento.

Se puede observar que al comienzo del proceso de entrenamiento, las imágenes generadas son completamente aleatorias. A medida que avanza el entrenamiento, el generador aprende la distribución de los datos reales y, a algunas épocas, algunos dígitos generados ya se asemejan a los datos reales.

```
In [18]: latent_space_samples = torch.randn(batch_size, 100).to(device=device)
generated_samples = generator(latent_space_samples)
```

```
In [19]: generated_samples = generated_samples.cpu().detach()
for i in range(16):
    ax = plt.subplot(4, 4, i + 1)
    plt.imshow(generated_samples[i].reshape(28, 28), cmap="gray_r")
    plt.xticks([])
    plt.yticks([])
```



```
In [20]: # Visualización del progreso de entrenamiento
# Para que esto se ve bien, por favor reinicien el kernel y corran todo el notebook

from PIL import Image
from IPython.display import display, Image as IImage

images = [Image.open(path) for path in list_images]

# Save the images as an animated GIF
gif_path = "animation.gif" # Specify the path for the GIF file
images[0].save(gif_path, save_all=True, append_images=images[1:], loop=0, duration=
display(IImage(filename=gif_path))
```

<IPython.core.display.Image object>

Las respuestas de estas preguntas representan el 30% de este notebook

PREGUNTAS:

- ¿Qué diferencias hay entre los modelos usados en la primera parte y los usados en esta parte?

En la primera parte, los modelos eran más profundos y trabajaban con vectores grandes, usando Dropout para regularización. Estaban diseñados para generar datos simples en 2D, las curvas. En esta segunda parte, los modelos son más pequeños y simples, con pocas capas y neuronas, porque la entrada y salida son vectores de solo 2 dimensiones. La regularización es mínima o nula.

- ¿Qué tan bien se han creado las imagenes esperadas?

En general, las imágenes generadas son muy buenas. Solo 2 de 16 imágenes son difíciles de interpretar, lo que indica que el modelo aprendió correctamente la distribución principal de los datos y genera muestras coherentes con la función objetivo.

- ¿Cómo mejoraría los modelos?

Se podría aumentar la capacidad de las redes agregando más neuronas o capas para capturar detalles mas específicos.

- Observe el GIF creado, y describa la evolución que va viendo al pasar de las épocas

Al principio, en las primeras 10 épocas, los puntos están todos desordenados y no se reconoce nada. Entre las épocas 10 y 30, empiezan a aparecer formas que parecen números, aunque todavía hay bastante ruido alrededor. Ya de la época 30 a la 50, los números se ven mucho más claros y ordenados, con menos puntos fuera de lugar.

```
In [21]: print()
print("La fraccion de abajo muestra su rendimiento basado en las partes visibles de
tick.summarise_marks() #
```

La fraccion de abajo muestra su rendimiento basado en las partes visibles de este laboratorio

70 / 70 marks (100.0%)